

B Appendix Method & Process

B.1 GitHub Issues

To keep an overview of the development process, GitHub's built-in Issues was used for project planning. GitHub issues is a built-in task management feature of GitHub, which allows for managing tickets and keeping track of development. The Issues feature lets owners of a repository create issues¹ for development to keep track of what needs to be done. To start development, issues were created based on each of the apps four main screens; Home, Explore, Saved, Account, to keep the screen as separated as possible. When creating an issue a short description, of what the issue was to fix, was attached. If an issue relied on the implementation of another issue it was marked as "blocked by [issue]" to keep an overview of what needs to be first. An example of an issue can be seen in following figure 1.

In addition to issues for development, the report relied on issues as well, to keep the writing process during development structured. Each issue was assigned to one person. GitHub Issues features a Kanban board which creates an overview of what is to be done, what is in progress, and what is done as seen in figure 4 of section 7.1. Because SpisNemt's GitHub repository is linked to GitHub issues it is possible to link branches to an issue during development, where branches can be created directly within the issue. When creating a pull request on a branch linked to an issue, the branch is automatically closed and moved to "Done".

¹An issue is a task which needs to be done, also called ticket or task in similar project management tools

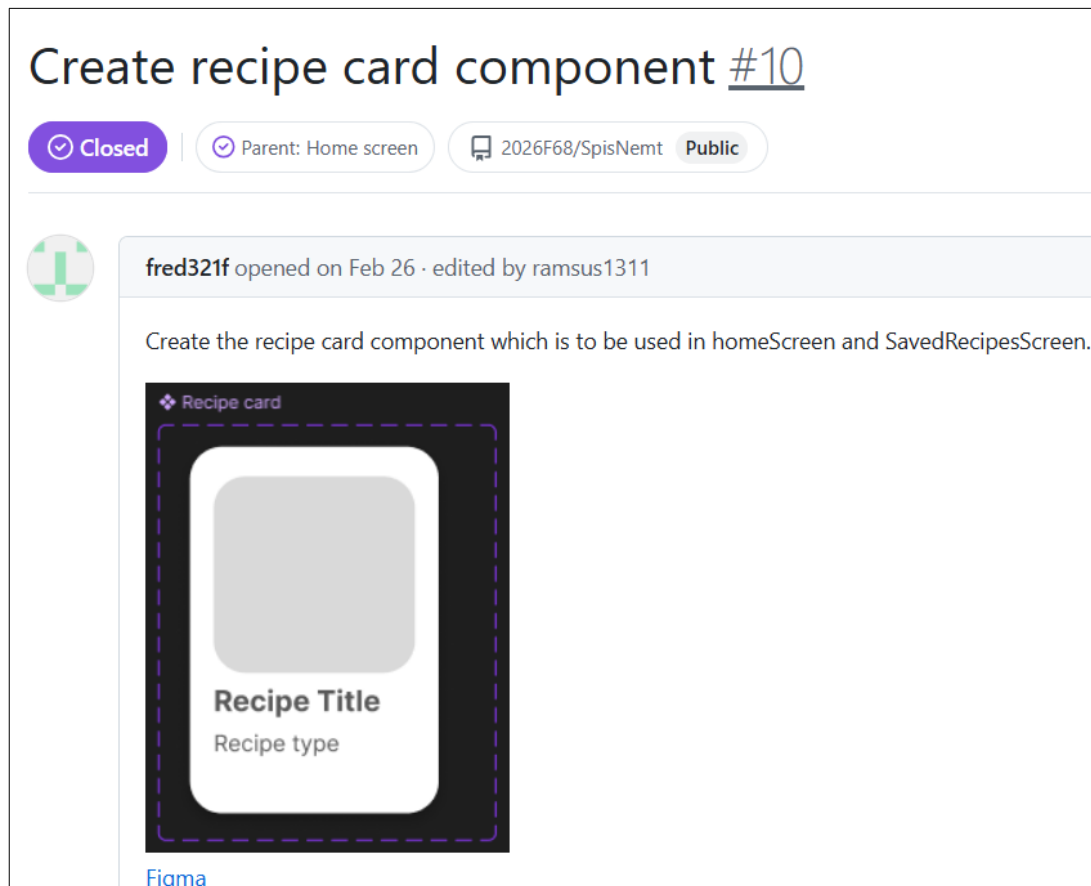
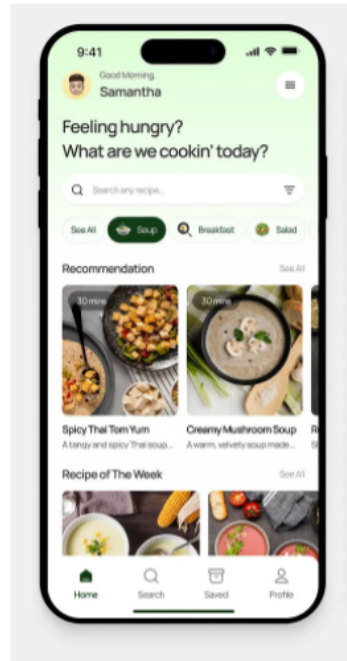


Figure 1: Example of a development ticket within the *GitHub Issues* interface, highlighting *Figma* design asset attachments, parent-epic tracking linkages, and issue identity metadata.

B.2 Moodboard

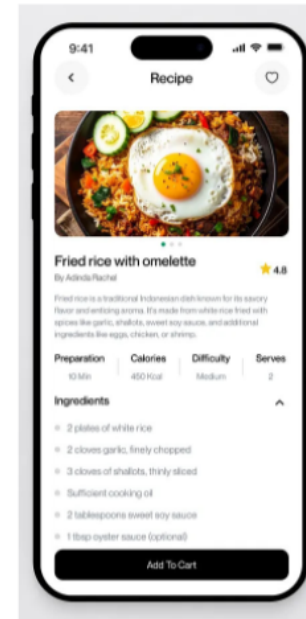
Before the architecture and design phase, inspiration was gathered from the website Dribbble, where food apps and alike was screenshotted and placed in a moodboard on Miro as seen in figure 2.



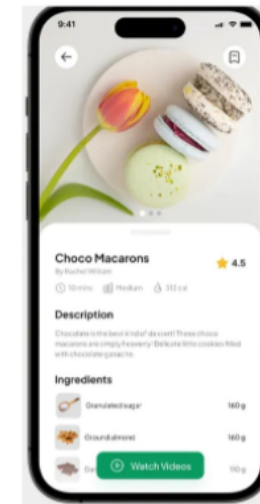
<https://dribbble.com/shots/26070104-Cookio-Food-Recipe-Mobile-App>



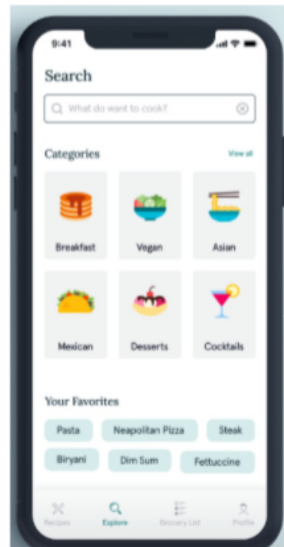
<https://dribbble.com/shots/19646794-Recipely-Food-App-Profile-Settings>



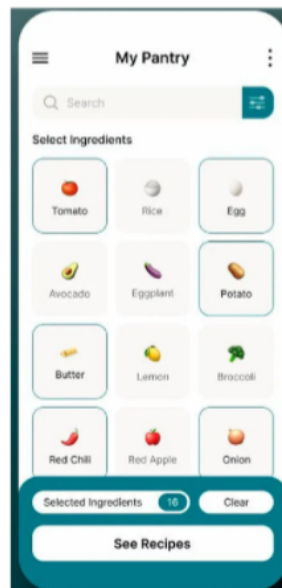
<https://dribbble.com/shots/27047103-Cooked-Food-Recipe-Mobile-App>



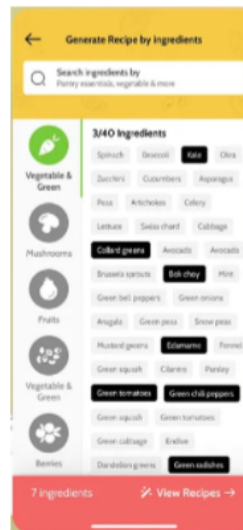
<https://dribbble.com/shots/20792040-Cookpedia-Food-Recipe-Mobile-App>



<https://dribbble.com/shots/14406965-Recipe-Search-Interaction>



<https://dribbble.com/shots/25336269-Recipe-App-Pantry-to-Plate>



<https://dribbble.com/shots/25733103-Smart-Recipe-App-UI>

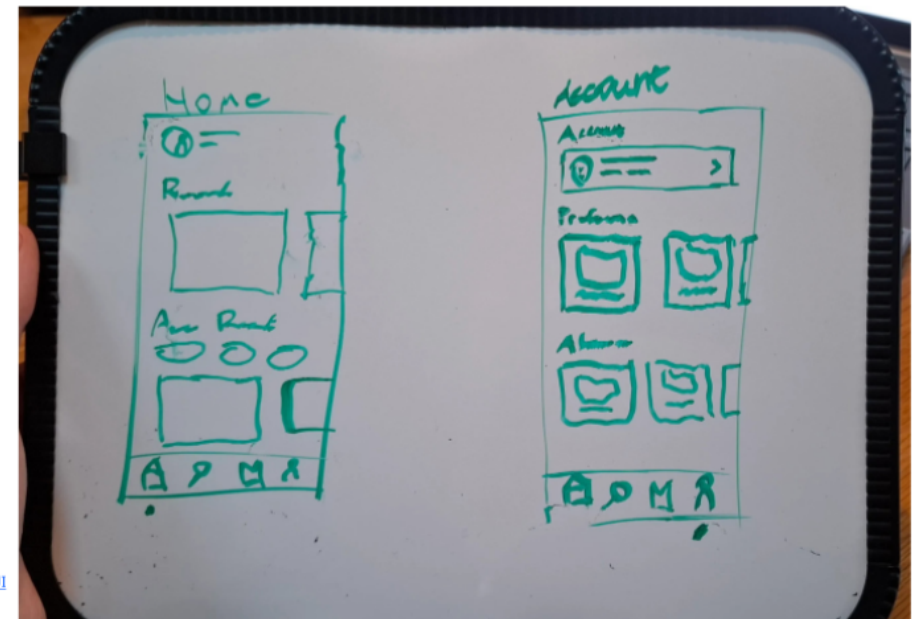


Figure 2: Moodboard on Miro with screenshots from dribbble

B.3 User interface design

As preparation to the development phase, low fidelity sketches were made in Figma² for the four most central screens as seen in figures 3, 4, 5, and 6. Low fidelity means a rough blueprint for the layout of the app, rather than a final design. The main driver while creating the low fidelity sketches was to design it with TypeScript components in mind as well as deciding what each screen contained. Components in front-end frameworks, such as Expo, are building blocks for the visual part of the app. A component can be a pillFilter or a card for recipes as seen in figure 3. A screen can be built up of multiple components layered on top of each other. Components can also have different variants if different color or sizing is needed. Components enable high modularity as they can be used in multiple places, which is a focus of SpisNemt. Another reason for creating low fidelity sketches of SpisNemt's four screens was so that all major design decisions were made before development started, so these two parts were kept apart.

²A collaborative, cloud-based design platform for building digital interfaces.

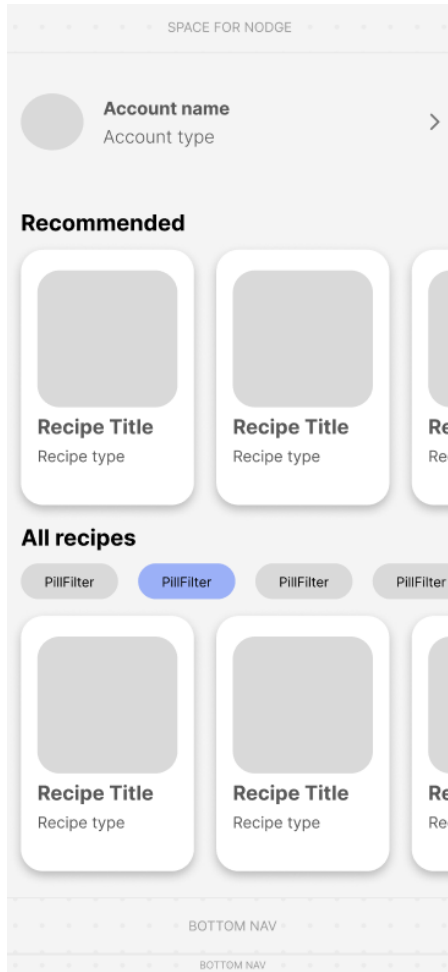


Figure 3: Home screen



Figure 4: Explore screen

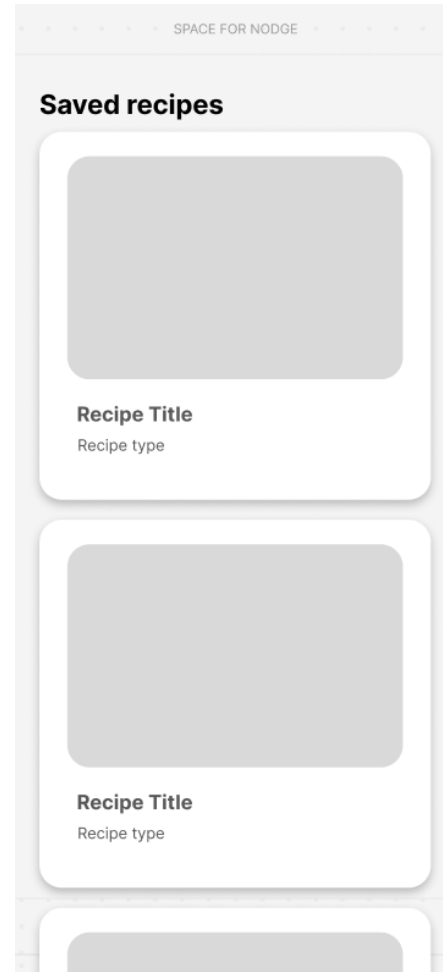


Figure 5: Saved recipes screen

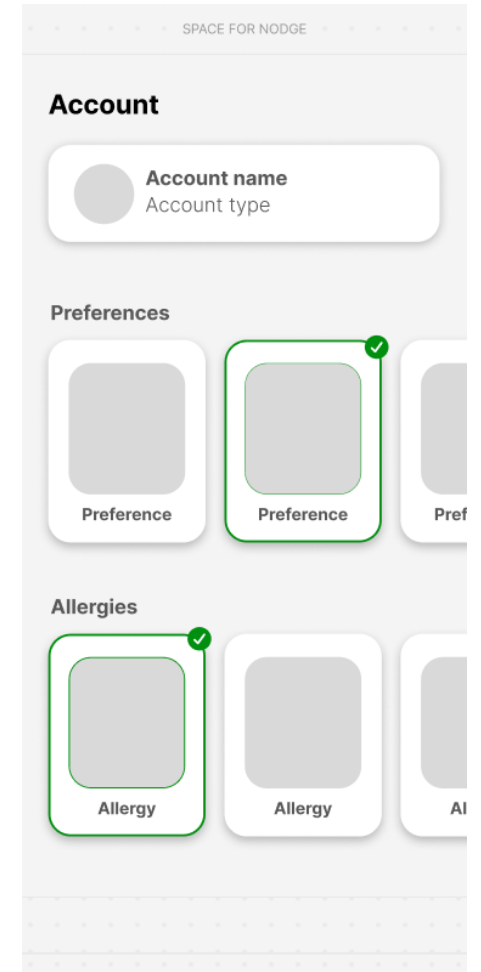


Figure 6: Account screen